

```
$ lab --phase 1 --num 3
```

Lab 3

IAM & S3 — Identité et stockage objet

Cours théorique + procédure de ce qu'on a fait

IAM	S3	Bucket	Policy	Role
-----	----	--------	--------	------

Région	Auteur	Date	Statut
eu-west-3 (Paris)	Nathan	Juin 2026	Validé ✓

COURS

Concepts théoriques

1. IAM — Identity and Access Management

IAM est le service qui répond à deux questions fondamentales dans AWS : **QUI** peut accéder à AWS, et **QUE** peut-il faire ? C'est le pilier sécurité d'AWS — tout repose dessus.

Les 4 concepts IAM

User	Une personne ou une application qui se connecte à AWS. A ses propres identifiants (login/mdp ou clés API).
Group	Un ensemble d'utilisateurs qui partagent les mêmes droits. Au lieu d'attacher des permissions à chaque user, on les attache au groupe.
Role	Une identité temporaire utilisée par des services AWS (EC2, Lambda...) et non par des humains. Pas de mot de passe — AWS gère l'attribution automatiquement.
Policy	Un document JSON qui décrit exactement ce qui est autorisé ou refusé. On attache les policies aux Users, Groups ou Roles.

La règle absolue — Deny gagne toujours

Si une policy **Allow** et une policy **Deny** s'appliquent toutes les deux sur la même action pour le même utilisateur, le Deny l'emporte systématiquement. Peu importe le nombre d'Allow, un seul Deny annule tout.

■ Implicit deny — tout est fermé par défaut

Par défaut dans AWS, tout est refusé (implicit deny). Si tu n'as pas de policy qui autorise explicitement une action, elle est refusée. Il faut donc toujours ajouter des permissions — jamais les retirer.

Anatomie d'une Policy JSON

Chaque policy contient des statements avec 3 éléments clés :

```
json
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Effect"   : "Allow",           ← Allow ou Deny
      "Action"   : "s3:GetObject",   ← ce qu'on peut faire
      "Resource" : "arn:aws:s3::mon-bucket/*" ← sur quoi
    }
  ]
}
```

Effect

Allow = autoriser / Deny = refuser. Deny gagne toujours sur Allow.

Action	L'opération autorisée. Format : service:action. Exemple : s3:GetObject, ec2:StartInstances, iam:CreateUser.
Resource	Sur quelle ressource s'applique la permission. Utilise le format ARN. '*' = toutes les ressources.
Principal	QUI a cette permission. Utilisé dans les bucket policies. '*' = tout le monde, ou un ARN spécifique.

2. Roles IAM — L'identité pour les services

Un rôle IAM c'est une identité qu'on **attache à un service AWS** (EC2, Lambda, etc.) pour lui donner des permissions. C'est la méthode recommandée — bien plus sécurisée que de stocker des clés API sur un serveur.

Clés API sur le serveur	Rôle IAM attaché à EC2
Clés stockées dans un fichier	Pas de clés — AWS gère l'auth
Si serveur compromis = clés volées	Si serveur compromis = le role s'arrête
Rotation des clés manuelle	Rotation automatique par AWS
Risque de clés dans le code source	Zero risque de fuite de clés

3. S3 — Simple Storage Service

S3 est le service de stockage de fichiers d'AWS. Tu stockes des **objets** (fichiers) dans des **buckets** (conteneurs). Chaque objet a une URL unique accessible depuis internet si tu l'autorises.

Caractéristiques clés

Durabilité	99.999999999% (11 neuf). AWS réplique tes fichiers dans 3 AZ automatiquement. Pratiquement impossible de perdre un fichier.
Scalabilité	Illimité. Tu peux stocker des pétaoctets sans rien configurer.
Nom unique mondial	Le nom de ton bucket doit être unique dans le monde entier AWS — pas juste dans ton compte.
Pas de hiérarchie	S3 n'a pas de vrais dossiers. Les '/' dans les noms de fichiers simulent une arborescence, mais c'est plat en réalité.

Block Public Access — Les 4 boucliers

AWS protège les buckets S3 par défaut avec 4 niveaux de blocage d'accès public. Pour héberger un site statique public, il faut désactiver les 2 derniers :

Block new ACL public access	Bloque les ACLs (ancien système de permissions). A garder activé — on n'utilise pas les ACLs.
Block any ACL public access	Même chose pour toutes les ACLs existantes. A garder activé.
Block new bucket policy public	Bloque les nouvelles bucket policies qui donnent un accès public. Peut être désactivé.
Block any bucket policy public	Bloque toutes les bucket policies publiques. DOIT être désactivé pour un site public.

■ Sécurité S3

Par défaut AWS bloque tout accès public aux buckets S3. C'est une protection contre les erreurs de configuration accidentelles — très fréquentes et parfois coûteuses (données sensibles exposées). Ne désactiver que ce qui est strictement nécessaire.

Bucket Policy — Contrôler l'accès

Une bucket policy est une policy IAM attachée directement au bucket (pas à un user). Elle définit qui peut faire quoi sur ce bucket depuis n'importe où.

```
json
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Effect" : "Allow",
      "Principal" : "*",          ← tout le monde (internet)
      "Action" : "s3:GetObject",  ← peut lire des fichiers
      "Resource" : "arn:aws:s3:::lab3-phase1-nathan-2701/*"
    }
  ]
}
```

4. Comment fonctionne l'accès EC2 → S3 via un Rôle

Quand une instance EC2 veut accéder à S3, voilà ce qui se passe en coulisses :

1. EC2 lance la commande	aws s3 ls s3://mon-bucket — l'instance veut accéder à S3.
2. AWS CLI interroge le metadata service	L'AWS CLI contacte automatiquement http://169.254.169.254/latest/meta-data/iam/security-credentials/ pour obtenir des credentials temporaires liés au rôle attaché à l'instance.
3. AWS fournit des credentials temporaires	AWS retourne un Access Key ID, Secret Access Key et un Session Token — tous temporaires (expiration automatique toutes quelques heures). Pas de clés stockées sur le disque.
4. La requête S3 est authentifiée	L'AWS CLI utilise ces credentials temporaires pour signer la requête vers S3. S3 vérifie les permissions du rôle → AmazonS3ReadOnlyAccess → accès autorisé.

■ Le metadata service AWS

Le metadata service (169.254.169.254) est une IP spéciale accessible uniquement depuis l'intérieur d'une instance EC2. C'est par lui que l'instance récupère toutes ses informations : son ID, sa région, son rôle IAM, ses credentials temporaires. Jamais accessible depuis internet.

PROCÉDURE

Ce qu'on a fait étape par étape

Étape 1 — Création du bucket S3

```
bash
Console AWS → S3 → Create bucket

Bucket name : lab3-phase1-nathan-2701
              (nom unique mondial – ajouter des chiffres si déjà pris)
AWS Region : eu-west-3

Block Public Access :
✓ Block public access to buckets and objects granted through new ACLs
✓ Block public access to buckets and objects granted through any ACLs
■ Block public access granted through new public bucket policies
■ Block public and cross-access through any public bucket policies
↑ Décocher les 2 dernières cases

→ Create bucket
```

Étape 2 — Upload du fichier et site statique

Créer un fichier index.html sur ton PC :

```
html
<!DOCTYPE html>
<html>
<head>
  <meta charset="UTF-8">
  <title>Mon premier S3</title>
</head>
<body>
  <h1>Hello depuis S3 !</h1>
  <p>Ce fichier est hébergé sur Amazon S3.</p>
</body>
</html>
```

```
bash
Bucket → Upload → Add files → index.html → Upload

Activer le site statique :
Bucket → Properties → Static website hosting → Edit
  Enable
  Index document : index.html
→ Save changes

URL du site : http://lab3-phase1-nathan-2701.s3-website.eu-west-3.amazonaws.com
```

Étape 3 — Bucket Policy pour l'accès public

```
Bucket → Permissions → Bucket policy → Edit

Coller cette policy :

{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Effect"    : "Allow",
      "Principal" : "*",
      "Action"    : "s3:GetObject",
      "Resource"  : "arn:aws:s3:::lab3-phase1-nathan-2701/*"
    }
  ]
}

→ Save changes
```

Étape 4 — Création du rôle IAM pour EC2

```
Console AWS → IAM → Roles → Create role

Trusted entity type : AWS service
Use case            : EC2
→ Next

Rechercher : S3ReadOnly
Cocher      : AmazonS3ReadOnlyAccess
→ Next

Role name : role-ec2-s3-lab-phase1
→ Create role
```

Étape 5 — Attacher le rôle à l'instance EC2

```
EC2 → Instances → ec2-web-lab-phase1
Instance state → Start instance (si stoppée)

Actions → Security → Modify IAM role
Sélectionner : role-ec2-s3-lab-phase1
→ Update IAM role
```


Étape 6 — Test de l'accès S3 depuis EC2

```
bash
# Depuis l'instance EC2 en SSH :
aws s3 ls s3://lab3-phase1-nathan-2701

# Résultat attendu (sans aucune clé configurée) :
2026-06-24 18:28:13    185    index.html
```

■ AWS CLI et rôle IAM

L'AWS CLI est pré-installé sur les AMI Ubuntu récentes d'AWS. Il détecte automatiquement le rôle IAM attaché à l'instance et utilise ses credentials temporaires pour s'authentifier.

Erreurs rencontrées

403 Forbidden sur l'URL du site statique

Cause : La 4ème case 'Block public and cross-account access through any public bucket policies' était encore cochée — elle bloquait la bucket policy.

Solution : Désactiver les 2 dernières cases de Block Public Access (garder seulement les 2 premières sur les ACLs).

Accents mal affichés (Ã© au lieu de é)

Cause : Le fichier HTML n'avait pas déclaré l'encodage UTF-8.

Solution : Ajouter dans le du fichier HTML.

VALIDATION

Checklist finale

✓	Bucket S3 créé	lab3-phase1-nathan-2701 en eu-west-3
✓	Block Public Access	2 dernières cases décochées
✓	Fichier uploadé	index.html présent dans le bucket
✓	Bucket Policy	GetObject public configuré
✓	Site statique	Static website hosting activé
✓	Site accessible	http://lab3-phase1-nathan-2701.s3-website.eu-west-3.amazonaws.com affiche la page
✓	Rôle IAM créé	role-ec2-s3-lab-phase1 avec AmazonS3ReadOnlyAccess
✓	Rôle attaché à EC2	ec2-web-lab-phase1 utilise le rôle
✓	Test aws s3 ls	Index.html visible depuis EC2 sans clé configurée
✓	Instance stoppée	EC2 stoppée pour économiser les crédits

■ Coûts

Coûts de ce lab : quasi nul. S3 stockage gratuit dans le Free Tier (5 GB). Le Static website hosting ne coûte que les requêtes (quelques centimes pour des tests). L'instance EC2 stoppée ne consomme que le disque EBS (quasi gratuit).